



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-46683 Snappy: SSRF and local file read via the xsl-style-sheet option

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-12
Version	1.0
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-46683 Snappy: SSRF and local file read via the xsl-style-sheet option - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-12 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

CVE-2026-46683 Snappy: SSRF and local file read via the xsl-style-sheet option

1. Executive Summary

CVE-2026-46683 affects KnpLabs Snappy, a PHP wrapper commonly used to generate PDFs, thumbnails, and snapshots through ``wkhtmltopdf``. The flaw sits in handling of the ``xsl-style-sheet`` option. In vulnerable releases prior to 1.7.0, attacker-controlled input can be treated as a retrievable URL and fetched by the application, enabling server-side request forgery (SSRF) and local file read behavior. Public CNA and NVD records score the issue as CVSS v4.0 6.9 / Medium, but practical risk can rise materially when Snappy is exposed through internet-facing workflows, runs with broad filesystem visibility, or can reach sensitive internal services.

The most important exposure pattern is not simple package presence, but unsafe application use of Snappy. The reviewed GitHub advisory explicitly shows a vulnerable pattern where untrusted request input is passed directly into ``xsl-style-sheet``. That makes this a dependency-plus-integration issue: organizations can miss it if they only track package versions without reviewing how PDF-generation features ingest user-controlled values.

Reviewed public sources do not show CISA KEV inclusion, public proof-of-concept weaponization, or confirmed in-the-wild exploitation as of this report date. However, the advisory, CVE publication, and patch details are all public and sufficiently specific for rapid adversary reproduction. Defenders should therefore prioritize package inventory, code-path review, and monitoring of PDF-generation workflows rather than treating the Medium base score as a reason to defer action.

Immediate defensive priorities are to upgrade Snappy to 1.7.0 or later, eliminate direct user control of ``xsl-style-sheet``, restrict outbound connectivity from PDF-rendering services, and hunt for suspicious ``wkhtmltopdf`` invocations or internal network access originating from PHP/PDF generation hosts.

2. Intelligence Requirement

Question	Answer
What is the vulnerability?	A SSRF and local file read issue in Snappy's handling of the <code>`xsl-style-sheet`</code> option.
Which product is affected?	KnpLabs <code>`snappy`</code> / Composer package <code>`knplabs/knp-snappy`</code> .
Which versions are affected?	Versions earlier than 1.7.0.
What is the fixed version?	Version 1.7.0.
What is the highest-risk exposure pattern?	Applications that pass untrusted user input into Snappy's <code>`xsl-style-sheet`</code> option while the PHP/PDF-rendering process can access sensitive local files or internal-only network resources.
What should defenders do first?	Patch to 1.7.0+, remove attacker control of stylesheet paths/URLs, and monitor for suspicious PDF-generation activity and internal egress.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Update Guide intake item	Intake trigger and publication tracking	Medium
CVE Program API record	Canonical publication dates, affected-version statement, CWE mapping, and	High

	CVSS v4.0 vector	
NVD CVE 2.0 API	Normalized vulnerability record, references, and severity restatement	High
GitHub advisory GHSA-c5fp-p67m-gq56	Human-readable exploit conditions, vulnerable code example, patch/workaround guidance	High
Snappy v1.7.0 release notes	Confirms the issue was remediated in 1.7.0 and links the security fix	High
Upstream patch for PR #548	Code-level evidence showing URL handling moved from broad URL validation to an allowlisted scheme model	High

Confidence is high that the vulnerability is real, affects Snappy versions below 1.7.0, and is tied specifically to unsafe handling of the ``xsl-style-sheet`` option. Confidence is medium on worst-case impact in a generic environment because public sources do not include a full exploit chain, target-specific proof of compromise, or broadly observed attacker tradecraft. One nuance from patch review is that version 1.7.0 improves default scheme handling, but safe deployment still depends on application-side allowlisting and service-level egress controls. In other words, patching is necessary, but developers should not interpret the fix as permission to accept arbitrary user-supplied stylesheet URLs.

4.1 Vulnerability Mechanics

The vulnerable Snappy code path evaluates option values and, for ``xsl-style-sheet``, fetches content when the supplied value is treated as a URL. In pre-1.7.0 code, URL detection used PHP's broad ``FILTER_VALIDATE_URL`` behavior. The reviewed patch replaces that with ``parse_url()`` plus an explicit allowlisted-scheme check, defaulting to ``http`` and ``https``, and adds a constructor parameter for allowed schemes.

This matters because the old behavior let an attacker-controlled ``xsl-style-sheet`` value reach a content retrieval path. In practice, the reviewed advisory and patch support two main abuse outcomes:

- SSRF, where the application fetches attacker-selected remote content from internal or otherwise unintended network destinations;
- local file read behavior, where non-HTTP URL schemes such as ``file://`` can be abused to coerce retrieval of local files into the rendering workflow.

4.2 Affected Scope

Dimension	Assessment	Evidence
Product	Knplabs Snappy	CVE Program API, NVD, GitHub advisory
Ecosystem	PHP / Composer (<code>`knplabs/knp-snappy`</code>)	GitHub advisory
Affected versions	<code>`< 1.7.0`</code>	CVE Program API, NVD, GitHub advisory
Fixed version	<code>`1.7.0`</code>	CVE Program API, release notes
Weakness	CWE-918 Server-Side Request Forgery (SSRF)	CVE Program API, NVD
Public severity	CVSS v4.0 6.9 / Medium	CVE Program API, NVD
CISA ADP enrichment	Exploitation: none; Automatable: yes; Technical impact: partial	CVE Program API ADP block

4.3 Exposure Patterns

Exposure Pattern	Why It Matters	Relative Risk
Internet-facing PDF/report generation endpoint with attacker-controlled stylesheet selector	Gives remote users a direct path into the vulnerable option handling logic	High
Internal application or admin workflow that accepts imported templates, report settings, or URL parameters	Abuse may still be possible via low-privilege insiders, compromised accounts, or chained web flaws	Medium
PHP service running with broad filesystem visibility or host-level access	Local file read consequences become more serious when sensitive config, keys, or secrets are reachable	High
Rendering tier with unrestricted outbound access to RFC1918/link-local destinations	Increases SSRF blast radius to internal web apps, metadata services, or management interfaces	High
Snappy installed but `xsl-style-sheet` never exposed to user influence	Package remains vulnerable in principle, but reachability is lower pending code review	Medium to Low

4.4 Patch Interpretation and Residual Risk

The upstream fix raises the baseline by replacing broad URL validation with an allowlisted-scheme approach and defaulting to `http`/`https`. That appears sufficient to block the most obvious `file:///`-based local read path by default. However, defenders should not treat the patch as a complete application-security boundary. If a workflow still allows arbitrary attacker-supplied HTTP/HTTPS URLs, the application may continue to reach internal services unless developers additionally constrain allowed values and operators enforce outbound network restrictions.

The reviewed GitHub advisory therefore points to two layers of remediation:

- upgrade to version 1.7.0 or later;
- remove direct user control over stylesheet selection by mapping user choices to a fixed internal allowlist.

5.1 Exploitation Status and Threat Outlook

No reviewed source indicates inclusion in CISA's Known Exploited Vulnerabilities catalog or confirmed in-the-wild exploitation as of 2026-06-12. The CVE record's CISA ADP enrichment explicitly marks exploitation as `none`, while also judging the issue automatable with partial technical impact. That profile is consistent with a vulnerability that is easy to reproduce once a reachable code path is found, but whose operational value still depends heavily on target-specific deployment choices.

From a threat-intelligence perspective, this issue is more dangerous than a generic Medium score may imply because it affects a common utility library embedded in business workflows that defenders do not always instrument deeply. Adversaries can also chain it with weak identity controls or existing web flaws: a low-privilege foothold into a report-generation feature may be enough to pivot into local secret discovery or internal service enumeration.

5.2 Representative Attack Scenarios

Scenario	Path	Defensive Implication
External attacker abuses a self-service PDF export function	Request parameter or JSON field is passed directly to `xsl-style-sheet`, causing the PHP worker to fetch attacker-chosen content	Prioritize internet-facing render/export features for code review and patching
Authenticated user or compromised	SSRF from the rendering host reaches	Restrict egress from render nodes and

account probes cloud metadata and internal apps	RFC1918 services or `169.254.169.254`	monitor internal-only destinations
Attacker targets sensitive local files	`file:///` or equivalent scheme is used to coerce local file retrieval in vulnerable versions	Patch quickly and review whether render hosts expose credentials, API tokens, or config files
Chained compromise in an internal portal	A separate web bug supplies the attacker with control over report settings that later invoke Snappy	Treat PDF/reporting components as security-sensitive, not as low-risk utility features

5.3 MITRE ATT&CK Mapping

The ATT&CK mapping below is scenario-dependent and should be interpreted as a likely exploitation pattern rather than a claim of observed adversary use.

Technique	Name	Rationale
T1190	Exploit Public-Facing Application	Fits internet-accessible workflows where user-supplied render settings reach Snappy.
T1005	Data from Local System	Appropriate when the local file read path is used to retrieve sensitive files from the host.
T1046	Network Service Discovery	Appropriate when SSRF is used to enumerate or probe internal services reachable from the rendering tier.

6.1 Detection Coverage Summary

Signal	Telemetry Source	Analytic Goal
Snappy package versions below 1.7.0	SBOM, container inventory, package inventory, Composer lockfiles	Identify latent exposure before exploitation occurs
`wkhtmltopdf` or PHP worker invocations containing `xsl-style-sheet`	EDR process telemetry, command-line logging, application debug logs	Surface the vulnerable code path in real environments
Outbound connections from render hosts to RFC1918, loopback, link-local, or metadata IP space	EDR network telemetry, firewall logs, proxy logs, VPC flow logs	Detect likely SSRF abuse
Requests or application logs containing `file:///`, `xsl-style-sheet`, or suspicious stylesheet URLs	Reverse proxy logs, WAF logs, app logs	Detect exploit attempts and user-controlled stylesheet usage
Access to sensitive local paths by rendering processes	EDR file events, auditd, Sysmon for Linux, application error logs	Detect possible local file-read exploitation or testing

6.2 Microsoft Defender KQL - Snappy / wkhtmltopdf Suspicious Process and Network Activity

<pre>let Lookback = 30d; let RenderProcesses = DeviceProcessEvents where Timestamp > ago(Lookback) where FileName in~ ("wkhtmltopdf", "php", "php-fpm", "php-cgi", "apache2") where ProcessCommandLine has_any ("xsl-style-sheet", "wkhtmltopdf", "Knp\\Snappy") project DeviceId, DeviceName, ProcTime=Timestamp, FileName, ProcessCommandLine, InitiatingProcessFileName, InitiatingProcessCommandLine; let SuspiciousNetwork = DeviceNetworkEvents</pre>
--

```

| where Timestamp > ago(Lookback)
| where InitiatingProcessFileName in~ ("wkhtmltopdf", "php", "php-fpm", "php-cgi", "apache2")
| where RemoteIPType in~ ("Private", "Loopback", "LinkLocal")
|   or RemoteIP in~ ("169.254.169.254", "127.0.0.1")
|   or RemoteUrl has_any ("localhost", "169.254.169.254")
| project DeviceId, NetTime=Timestamp, RemoteIP, RemoteUrl, RemotePort, InitiatingProcessFileName,
InitiatingProcessCommandLine;
RenderProcesses
| join kind=leftouter SuspiciousNetwork on DeviceId
| where isnotempty(RemoteIP) or ProcessCommandLine has "xsl-style-sheet"
| project ProcTime, NetTime, DeviceName, FileName, ProcessCommandLine, RemoteIP, RemoteUrl, RemotePort,
InitiatingProcessFileName, InitiatingProcessCommandLine
| order by coalesce(NetTime, ProcTime) desc

```

6.3 Sigma - Suspicious wkhtmltopdf Stylesheet Retrieval

```

title: Suspicious wkhtmltopdf xsl-style-sheet Usage
id: 79fe1f67-f941-4d6d-8f94-cve-2026-46683
status: experimental
description: Detects use of wkhtmltopdf or related PHP rendering workflows with suspicious xsl-style-sheet arguments that may indicate exploitation or unsafe application behavior.
logsource:
  product: linux
  category: process_creation

detection:
  selection_binary:
    Image|endswith:
      - '/wkhtmltopdf'
      - '/php'
      - '/php-fpm'
  selection_option:
    CommandLine|contains: 'xsl-style-sheet'
  selection_target:
    CommandLine|contains:
      - 'http://'
      - 'https://'
      - 'file://'
      - '169.254.169.254'
      - '127.0.0.1'
      - 'localhost'
  condition: selection_binary and selection_option and selection_target
falsepositives:
  - Legitimate internal stylesheet retrieval during approved rendering workflows
  - Security testing or QA validation of report-generation code
level: high
tags:
  - attack.t1190
  - attack.t1005
  - attack.t1046

```

6.4 Hunting Leads

Hunt Hypothesis	Data to Review	Expected Analyst Outcome
A user-facing report/export function exposes the vulnerable option	Application routes, request handlers, form parameters, API schemas	Confirm whether user-controlled input can reach `xsl-style-sheet`
Render hosts were used to access internal services	Network telemetry from PHP and `wkhtmltopdf` processes	Identify SSRF reachability and suspicious internal destinations
Attackers attempted local file retrieval	Process command lines, app errors, file telemetry, WAF logs	Identify `file://`-style abuse and sensitive path targeting
Exposure persists inside vendored	`composer.lock`, container layers, deployed	Find assets missed by OS-level package

dependencies	vendor directories	inventory
--------------	--------------------	-----------

6.5 Splunk Hunt Query - Web and Application Evidence of Stylesheet Abuse

```
(index=web OR index=proxy OR index=app_logs)
("xsl-style-sheet" OR "wkhtmltopdf" OR "Knp\\Snappy" OR "file://" OR "169.254.169.254")
| eval suspected_target=coalesce(url, uri, request, message)
| stats count min(_time) as first_seen max(_time) as last_seen values(host) as hosts values(src) as src_ips values(suspected_target) as targets by user, app, sourcetype
| convert ctime(first_seen) ctime(last_seen)
| sort - count
```

7. Recommendations

Priority	Owner	Recommendation	Rationale
Immediate	Application Owners / Engineering	Upgrade `knplabs/knp-snappy` to version 1.7.0 or later everywhere it is deployed.	Removes the known vulnerable pre-1.7.0 URL-handling behavior.
Immediate	Application Owners	Eliminate direct user control of `xsl-style-sheet`; map user choices to a fixed internal allowlist instead.	Advisory guidance indicates the vulnerable pattern is direct pass-through of attacker input.
Immediate	Platform / Cloud Operations	Restrict outbound connectivity from PDF/rendering services to only required destinations and block metadata/IP ranges where feasible.	Reduces SSRF blast radius even if an application flaw remains reachable.
High	Detection Engineering / SOC	Deploy hunts for `wkhtmltopdf` or PHP command lines containing `xsl-style-sheet`, `file://`, metadata IPs, or internal-only targets.	Creates near-term visibility into both exploitation attempts and unsafe application design.
High	Vulnerability Management	Search `composer.lock`, container images, and vendored application paths for `knplabs/knp-snappy` versions below 1.7.0.	Library issues are often missed when only system packages are inventoried.
Medium	Security Architecture	Treat reporting/PDF generation features as high-risk web components during threat modeling and code review.	Utility rendering paths frequently gain broad filesystem and network access.
Medium	Incident Response	If suspicious activity is found, preserve request samples, application logs, command lines, and network traces from the rendering host before cleanup.	Evidence quality matters for distinguishing unsafe implementation from attempted exploitation.

Additional analyst notes:

- Do not down-prioritize this issue solely because the public base severity is Medium; business risk becomes High quickly when render hosts can access sensitive internal resources.

- Patching is necessary but not sufficient. Secure-by-design handling requires both application allowlisting and network egress controls.
- Where immediate code changes are not possible, place compensating controls around PDF/report-generation features and increase alerting on internal egress from those services.

8. References

- Microsoft Security Update Guide intake item, `CVE-2026-46683`
- <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-46683>
- CVE Program API record, `CVE-2026-46683`
- <https://cveawg.mitre.org/api/cve/CVE-2026-46683>
- NVD CVE 2.0 API, `CVE-2026-46683`
- <https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-46683>
- GitHub Advisory Database, `GHSA-c5fp-p67m-gq56`
- <https://github.com/KnpLabs/snappy/security/advisories/GHSA-c5fp-p67m-gq56>
- Snappy release `v1.7.0`
- <https://github.com/KnpLabs/snappy/releases/tag/v1.7.0>
- Upstream patch for PR `#548`
- <https://patch-diff.githubusercontent.com/raw/KnpLabs/snappy/pull/548.patch>